

Analytics Handbook

How to get into PostHog, confirm data is arriving, and answer questions with it — activity, persons, filters, cohorts, insights and SQL. Plus what Unity Gaming Services is for and when you need to touch it.

Event pipeline	One funnel in the game fans out to two destinations: UGS Analytics and PostHog
PostHog region	EU Cloud (Frankfurt) — <code>eu.posthog.com</code>
Identity	Every person is a UGS Player ID, searchable by display name
Collection	Opt-in — only players who accept the in-game privacy dialog appear
Schema version	1 — carried on every event as <code>schema_version</code>

1 Getting in

Free tier, unlimited seats — everyone who analyses gets full access

- 1 Go to eu.posthog.com and sign in. **The EU address matters** — a US login will show an empty project, because ours is EU Cloud and the region cannot be changed.
- 2 Open the **Cosmic Shore** project.
- 3 Need access? Ask for an invite: **Organization settings** → **Members** → **Invite**. The free tier has unlimited members, so there is no reason to share a login.

Everything below lives in the left-hand nav. The four you will actually use:

NAV ITEM	WHAT IT IS FOR
Activity	The raw live feed. Use it to confirm data is arriving and to inspect one event's properties.
People	One row per player. Full event history, person properties, and Cohorts.
Product analytics	Trends, Funnels, Retention, and saved insights / dashboards.
SQL	HogQL editor for anything the built-in insights cannot express.

Only consenting players appear

Collection is opt-in. The game shows an age gate and a consent dialog on first run, and a player who declines — or who is under 13 — is never collected at all. So PostHog shows the *consenting* population, not everyone who played. That is deliberate and legally required; treat totals as a floor, not a headcount.

2 Is data arriving?

Start here whenever something looks wrong

- 1 Open **Activity**. Events appear within seconds of a flush.
- 2 Look for `game_started` and `game_completed`.
- 3 Click any event to expand its properties.

Events send in batches, not instantly

The game queues events and uploads on a batch size or interval, and always on pause/quit. If you are testing, **quit the game** and the queue flushes. A process killed outright can lose whatever had not been sent.

What is on every event

PROPERTY	MEANING
distinct_id	The person. Equals the UGS Player ID — the join key across PostHog, UGS and Cloud Save.
player_id	Same value, carried explicitly so it can be filtered as a property.
install_id	Device-scoped ID. Survives sign-out, so one device can be followed across accounts.
app_version	Build version. Always filter by this when comparing before/after a change.
platform	Windows, Android, editor, and so on.
schema_version	Event-contract version. Lets old rows be told apart from new ones after a change.

The two events that carry the most

`game_started` — one per game, at the moment players are given control:

PROPERTY	MEANING
match_id	One game instance. Every player in the same match shares it.
party_id	One sitting. Stays the same across back-to-back games by the same party.
player_ids	Array of the human players in the match. AI excluded.
invite_triggered	True if the party formed via a formal invite; false if players met through the lobby.
player_count_human / _ai	Humans, and AI backfill.
game_mode · intensity · vessel_class	What was played, how hard, in what.

`game_completed` — one per game, at the end:

PROPERTY	MEANING
flight_time_seconds	Time actually at the controls. Pause and time in the background are excluded.
duration_seconds	Wall clock, pause included.
timestamp_utc_ms / _iso	Client clock at completion, for day/week/month bucketing.
match_id · party_id	Echoed, so completion joins back to the start.
player_won	Whether the local player was on the winning side.

A free churn signal

`duration_seconds - flight_time_seconds` is pause plus idle plus between-round time. A game where that gap is large is a game people are sitting through rather than playing.

Nothing showing up at all?

In order of likelihood: the player never accepted the consent dialog; the app was killed before a flush; the event name is in the project's *Excluded Events* list; or there was no network (events queue and retry, but a killed process loses the queue).

3 People and person properties

Who a player is, and what is true about them right now

Open **People**. One row per player. Two ways to find someone:

- **By UGS Player ID** — the canonical identity, and what `distinct_id` is.
- **By display name** — search matches any person property, so typing a player's in-game name finds them.

Search by name, but never group by it

Display names are mutable and not unique — two players can pick the same one, and a rename would split one player into several. The ID is the identity; the name is a convenience for finding a human you already know about.

Person properties

These are the player's *current* state, mirrored from their save data and overwritten as it changes. Filter or build cohorts on any of them.

PROPERTY	MEANING
<code>display_name</code>	In-game name.
<code>first_seen_utc_ms</code>	When the account was created — the basis for cohorting by join date.
<code>session_count</code>	How many sessions they have played.
<code>games_completed</code>	Lifetime completed games.
<code>total_flight_time_seconds</code>	Lifetime time at the controls.
<code>preferred_vessel</code>	Their most-flown vessel — derived, not chosen.
<code>selected_vessel</code>	The vessel they last picked.
<code>unlocked_vessel_count</code>	How many vessels they own.
<code>crystal_balance</code>	Current currency. With the two lifetime totals below, this shows hoarding vs spending.
<code>lifetime_crystals_earned</code> / <code>_spent</code>	Lifetime economy totals.

Current value, not history

Person properties are overwritten. `crystal_balance` is what they hold now, not what they held last week. For change over time use the *events* (`crystals_earned`, `crystals_spent`), which are immutable.

4 Filters

The same control in every view

Every view — Activity, People, and each insight — has the same filter bar. Add a condition with **+ Filter**, then choose what to filter on:

FILTER ON	USE IT FOR
Event properties	Facts about one event: <code>game_mode = HexRace</code> , <code>intensity = 3</code> , <code>flight_time_seconds > 300</code> .
Person properties	Facts about the player: <code>games_completed > 10</code> , <code>preferred_vessel = Squirrel</code> .
Cohorts	A saved group of people (section 5).

Conditions combine with AND. For OR, either use a cohort or drop into SQL.

Three worth saving

- **Real sessions only** — `flight_time_seconds > 60` on `game_completed`, to exclude instant quits.
- **One build** — `app_version = <your build>`. Essential when comparing before and after a change; without it you are averaging across builds.
- **Real players only** — `platform != WindowsEditor`, to exclude the team's own editor sessions.

Your own testing is in the data

Editor play sessions are collected like any other. When measuring player behaviour, filter the editor out or your own testing will dominate a small dataset.

5 Cohorts

A named group of people you can reuse anywhere

A cohort is a saved definition of a group. Build it once, then filter any insight by it.

- 1 Go to **People** → **Cohorts** → **New cohort**.
- 2 Name it something a colleague would understand in six months.
- 3 Add matching criteria — a person property, or having performed an event.
- 4 Choose the type, then **Save**.

TYPE	BEHAVIOUR	USE WHEN
Dynamic	Recalculates — people join and leave as they qualify.	Almost always. "Active players", "whales", "new this week".
Static	Frozen at creation.	You need a fixed list, e.g. everyone in one playtest weekend.

Cohorts worth having for the playtest

COHORT	DEFINITION	ANSWERS
Engaged	Completed <code>game_completed</code> at least 3 times	Who actually stuck with it
One and done	Did <code>game_started</code> once, never <code>game_completed</code>	Where first-time players fall off
Social	Performed <code>party_joined</code> at any time	Whether playing together changes retention
Frustrated	Performed <code>repeated_game_fail</code>	Whether losing streaks predict quitting
Per-vessel	<code>preferred_vessel = <vessel></code>	Whether vessel choice changes how much people play

The pattern that makes cohorts pay

Define a cohort by *one* behaviour, then look at a *different* metric through it. Take the Social cohort and open a Retention insight filtered to it: that is a direct read on whether playing with other people brings players back — the question this instrumentation was commissioned to answer.

6 Insights

Most questions need no SQL

Product analytics → **New insight**. Prefer these over SQL — they handle sessionization and cohorting for you.

QUESTION	INSIGHT TYPE	HOW TO SET IT UP
Daily / weekly active players	Trends	Event <code>game_started</code> , measured by <i>unique users</i>
Which modes get played	Trends	<code>game_completed</code> , broken down by <code>game_mode</code>
Where new players drop off	Funnels	<code>game_first_launched</code> → <code>menu_ready</code> → <code>freestyle_entered</code> → <code>game_started</code> → <code>game_completed</code>
Do they come back?	Retention	First event <code>game_first_launched</code> , returning event <code>game_started</code>
How long is a session really	Trends	<code>game_completed</code> , average of <code>flight_time_seconds</code>

Save anything useful and pin it to a dashboard. A dashboard is the shared team view — otherwise every question gets rebuilt from scratch by whoever asks it next.

7 SQL (HogQL)

For everything the insight builder cannot express

SQL in the left nav, or **New insight** → **SQL**. The `events` table has `event`, `timestamp`, `distinct_id`, and `properties.*`. Property values read as JSON — wrap numbers in `toInt()` or `toFloat()` before aggregating.

```
-- Daily active players, last 30 days
SELECT toDate(timestamp) AS day, count(DISTINCT distinct_id) AS dau
FROM events
WHERE timestamp > now() - INTERVAL 30 DAY
GROUP BY day ORDER BY day
```

```
-- Time actually spent flying, per mode
SELECT properties.game_mode AS mode,
       round(avg(toFloat(properties.flight_time_seconds))) AS avg_flight_s,
       round(avg(toInt(properties.duration_seconds)))      AS avg_wall_s,
       count() AS games
FROM events
WHERE event = 'game_completed'
GROUP BY mode ORDER BY games DESC
```



```
-- Win rate by mode and intensity (multiplayer balance)
SELECT properties.game_mode AS mode,
       toInt(properties.intensity) AS intensity,
       round(countIf(toString(properties.player_won) = 'true') / count(), 3) AS win_rate,
       count() AS games
FROM events
WHERE event = 'game_completed' AND properties.player_won IS NOT NULL
GROUP BY mode, intensity ORDER BY games DESC
```

```
-- What players were doing right before they quit (churn forensics)
SELECT properties.last_ui_action AS last_action, count() AS sessions
FROM events
WHERE event = 'session_ended'
GROUP BY last_action ORDER BY sessions DESC
```

```
-- Crystal economy: earned vs spent per day
SELECT toDate(timestamp) AS day,
       sumIf(toInt(properties.amount), event = 'crystals_earned') AS earned,
       sumIf(toInt(properties.amount), event = 'crystals_spent') AS spent
FROM events
WHERE event IN ('crystals_earned', 'crystals_spent')
GROUP BY day ORDER BY day
```

The rematch question

This is what `match_id`, `party_id`, `player_ids` and `invite_triggered` were added for: **do groups voluntarily come back and play together again?**

```
-- Every group session: who played together, when, and whether an invite prompted it
SELECT properties.party_id           AS party,
       properties.match_id           AS match,
       properties.player_ids          AS players,
       properties.game_mode           AS mode,
       toString(properties.invite_triggered) AS was_invited,
       min(timestamp)                 AS played_at
FROM events
WHERE event = 'game_started'
      AND toInt(properties.player_count_human) > 1
GROUP BY party, match, players, mode, was_invited
ORDER BY played_at DESC
```

Reading the result

An **organic rematch** is the same `players` set, same `mode`, `was_invited = false`, in a **different party**, within your chosen window.

The different-`party_id` condition is the one that is easy to miss and matters most. A party that plays three games back to back keeps one `party_id` — without that condition, one evening reads as two rematches. `match_id` separates the games; `party_id` separates the sittings.

Rematch rate per mode = organic rematches ÷ distinct groups who played that mode. That is the number that says which modes make people want to come back together.

8 Unity Gaming Services

The system of record — and it needs setting up before it shows anything

Every event goes to **both** PostHog and UGS Analytics. They serve different jobs:

	POSTHOG	UGS ANALYTICS
Use for	Exploration: cohorts, funnels, retention, ad-hoc SQL	System of record, and the same identity as Cloud Save and Leaderboards
Accepts	Anything the game sends	Only events defined in its dashboard first
Where	<code>eu.posthog.com</code>	<code>cloud.unity.com</code> → Analytics

Why UGS can read zero while PostHog has data

UGS validates every custom event against a schema created by hand in its dashboard. Anything not defined there is **rejected**, not stored. So until the setup below is done, UGS shows 0 valid events no matter how much the game sends. PostHog is unaffected — it accepts whatever arrives, which is why it is the faster place to check that the pipeline works.

Setting it up — two passes, in this order

[cloud.unity.com](#) → [Development](#) → [Products](#) → [Analytics](#) → [Event Manager](#)

- 1 Create the parameters first (42 of them).** Parameters are project-level objects shared across environments — you create one once and then attach it to every event that uses it. `game_mode` alone is used by seven events.
- 2 Then create the events (28) and assign parameters to each.** Doing it the other way round means stopping mid-event to go and create a parameter that does not exist yet.

Both lists, with names, types and descriptions already written, are in the repo:

`Docs/Analytics/ugs_step1_parameters.csv` and `ugs_step2_events.csv`. Descriptions are mandatory in the dialogs.

Nothing here can be deleted

UGS events and parameters cannot be removed once created, and they count against a limit of 1,500 parameters per environment. Names are case-sensitive. Get them right the first time — copy from the CSVs rather than retyping.

Checking UGS

- Event Manager** shows valid and invalid event counts for the last 24 hours.
- Event Browser** → **Invalid Events** shows what was rejected and why. If UGS looks empty but the game is definitely sending, look here first — the events are usually arriving and being turned away.

Cloud Save — the other half of the data

Progression, currency, vessel stats and per-mode records live in Cloud Save, not in events. They can be exported in bulk with the scripts in `Tools/Analytics/`. The exported `player_id` is the same value as the PostHog `distinct_id`, so save data and event data join directly with no mapping table.

9 Things that will confuse you

Read once, save an afternoon

SYMPTOM	WHAT IS ACTUALLY HAPPENING
Both PostHog and UGS are empty	Nothing to do with either backend. The player has not accepted the consent dialog, so the game drops every event before it is sent. Consent is opt-in and defaults to denied.
PostHog has data, UGS reads 0 valid	Normal until the UGS Event Manager setup (section 8) is finished. UGS rejects undefined events.
Fewer players than took part in the playtest	Only consenting players are collected. Expect the total to be lower than the invite list.
Numbers look odd on a small dataset	Team editor sessions are in there too. Filter <code>platform != WindowsEditor</code> .
An event is missing right after play	Batching. It sends on pause/quit or the next interval — quit the game and check again.
Two players with the same name	Display names are not unique. Use the ID.
<code>player_won</code> missing on some games	Not every mode resolves a winner. Filter it out rather than treating absent as a loss.

Deleting a player's data

If someone asks to be removed: PostHog → **People** → find them by UGS Player ID → **Delete person**, including their events. The in-game button flags the request but cannot complete it, because the client only holds a write-only key. Treat this as a manual step until the server-side path is built.

Reference material in the repo: `Docs/Analytics/ANALYTICS_HANDOFF.md` (status and open actions) · `DATA_ARCHITECTURE.md` (schema design) · `EVENT_SCHEMA.json` (the full event contract) · `POSTHOG_SETUP.md` (project setup) · `Tools/Analytics/README.md` (Cloud Save export).